The Perceptron: Model, Neuron and Geometry

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. The parts of a perceptron . . . 3
   - 3.1 Inputs, weights and bias . . . 3
   - 3.2 The summation . . . 3
   - 3.3 The activation function . . . 4
   - 3.4 The bias as a threshold . . . 4
4. Training and prediction . . . 5
   - 4.1 More inputs . . . 7
5. Perceptron and biological neuron . . . 7
   - 5.1 Three differences . . . 8
6. Weights as feature importance . . . 8
7. What a perceptron does geometrically . . . 10
   - 7.1 The big limitation . . . 11
8. A perceptron in scikit-learn . . . 11
9. Summary . . . 14
10. Sources . . . 14
11. Key terms . . . 15

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Dot product; Equation of a hyperplane; Perceptron trick; Logistic regression.
- **Leads to:** Problem with the perceptron (XOR); Multi-layer perceptron (MLP); Activation functions.

1. Overview

Key point: A perceptron multiplies each input by a weight, adds a bias, and passes the sum through an **activation function** (G-165). With a step activation it defines one hyperplane (a straight line for two inputs) and puts each class on one side.

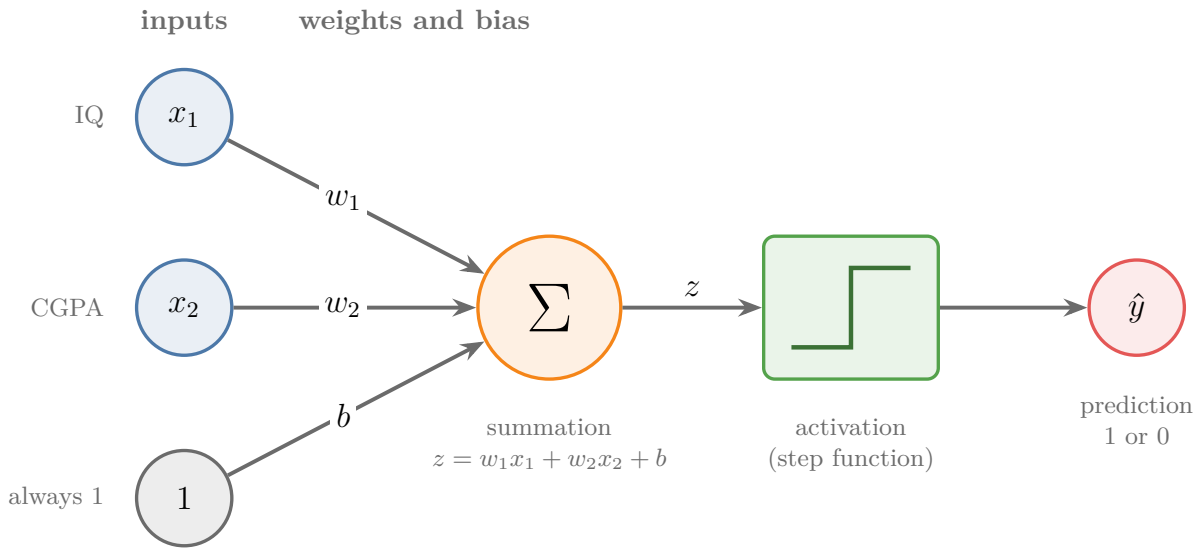


Figure 1: A perceptron with two inputs

The **perceptron** (G-1486) is the building block of every **neural network** (G-1316; see deep learning as ML with neural networks). It is also a **supervised learning** (G-1919) algorithm in its own right, like linear regression or logistic regression. Figure 1 shows its whole design.

This Note covers six things:

- the parts of the model (section 3);
- training and prediction (section 4);
- how it compares with a brain cell (section 5);
- what the weights say about each input (section 6);
- what it does geometrically (section 7);

- training one in scikit-learn (section 8).

How the perceptron learns its weights starts with the perceptron trick.

2. Prerequisites

- The general form of a line, its positive and negative sides, and the **step function** (G-1889): the **perceptron trick** (G-1485), sections 3 and 7.
- Lines, **planes** (G-1502) and hyperplanes: the **equation of a hyperplane** (G-703).
- The dot product: the **dot product** (G-634).

3. The parts of a perceptron

> **Key point:** Inputs, weights, a bias, a summation block that computes z, and an **activation function** (G-165) that turns z into the output.

3.1 Inputs, weights and bias

> **Key point:** Each input has its own weight; the bias is the weight on a constant input of 1.

In Figure 1, the data enters on the left. Each **feature** (G-772; an input variable, one column of the data table) gets one input node: x_1 and x_2.

- The connection from each input carries a **weight** (G-2106; w_1, w_2; see the parts of a neural network). The weight says how strongly that input counts.
- One extra input is always 1. Its connection carries the **bias** (G-284) b. The bias sets how easily the **perceptron** (G-1486) outputs 1, and it lets the line sit anywhere, not only through the origin (section 3.4).

The weights and the bias are the numbers the perceptron learns. Together they are its parameters (G-1448): the numbers a trained model is described by.

3.2 The summation

> **Key point:** z is the dot product of the weights with the inputs, plus the **bias** (G-284).

The orange block computes the **summation** (G-1915), a **weighted sum** (G-2119) of the inputs:

1. **In words:** multiply each input by its weight, add them up, and add the bias.
2. **Formula:**
$$z = w_1 x_1 + w_2 x_2 + b$$
3. **Example:** with $w_1 = 1$, $w_2 = 2$, $b = 3$ and inputs $x_1 = 100$, $x_2 = 5.1$, the bias input is always 1:
$$w_1 x_1 = 1 \times 100 = 100$$

$$w_2 x_2 = 2 \times 5.1 = 10.2$$

$$b \times 1 = 3 \times 1 = 3$$

$$z = 100 + 10.2 + 3 = 113.2$$

The sum $w_1 x_1 + w_2 x_2$ is the **dot product** (G-634) of the weight vector with the input vector (a vector is a list of numbers, here (w_1, w_2) and (x_1, x_2); see the **dot product** (G-634)).

3.3 The activation function

> **Key point:** The activation function maps z, which can be any number, into a fixed range. The **perceptron** (G-1486) uses the step function: 1 if z $\geq$ 0, else 0.

z can be any number, large or small, positive or negative. The green block in Figure 1 is the **activation function** (G-165): a function that brings z into a fixed range.

The classic perceptron uses the step function (G-1889): output 1 when $z \geq 0$ and 0 otherwise. Other activation functions exist (sigmoid, tanh, ReLU and more), with ranges such as 0 to 1 or -1 to 1. They are compared in activation functions.

3.4 The bias as a threshold

> **Key point:** The **perceptron** (G-1486) outputs 1 when the weighted sum of the inputs is at least $-$b. So the bias sets how large the weighted sum must be before the output switches to 1.

In plain words, the **bias** (G-284) decides how easily the perceptron says 1. A large positive bias makes it say 1 for almost any input; a large negative bias makes it hard to convince.

The standard term for this switching point is the **threshold** (G-2261). The steps:

1. The **step function** (G-1889) outputs 1 when $z \geq 0$.
2. z is the **weighted sum** (G-2119) plus the bias: $z = (w_1 x_1 + w_2 x_2) + b$.
3. So the output is 1 exactly when $w_1 x_1 + w_2 x_2 \geq -b$. The threshold is $-b$.

For example, with $b = -10$ the perceptron outputs 1 only when the weighted sum reaches 10.

Figure 2 shows the threshold on real data: the perceptron trained in section 8 on 100 students. Its score is

$$z = 5.82\, x_1 + 1.48\, x_2 + b$$

The weights stay as they are and only b changes.

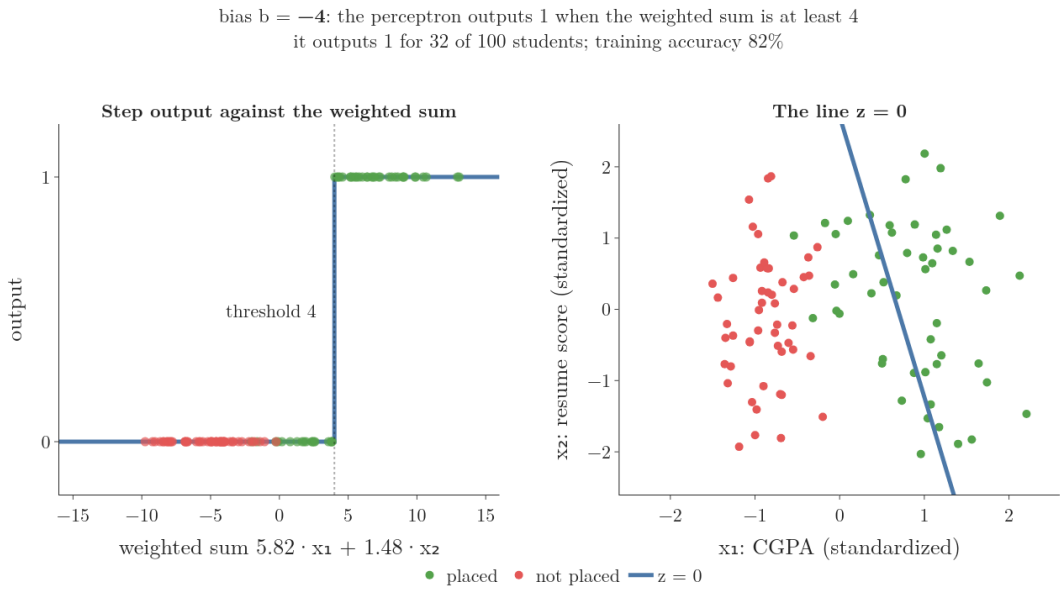


Figure 2: The bias as a threshold. Left: the step output against the weighted sum; the jump sits at $-b$. Right: the same perceptron on the input plane; the line $z = 0$ moves sideways without turning. The bias goes from 4 down to -4.

Bias b	Threshold $-b$	Students given output 1	Training accuracy
4	-4	62 of 100	88 percent
1 (the trained value)	-1	49 of 100	97 percent
-4	4	32 of 100	82 percent

- **Left panel:** as b falls, the jump of the step function slides to the right, and fewer students pass it.
- **Right panel:** the same change, seen on the two inputs. The line $z = 0$ keeps its direction, because the weights set the direction. The bias only moves the line parallel to itself. With $b = 0$ the line passes through the origin, so without a bias the perceptron could only draw lines through the origin.

4. Training and prediction

> **Key point:** Training finds the weights and **bias** (G-284) from labelled data. Prediction puts a new **observation** (G-1374; one record, one row of the data table) through the **perceptron** (G-1486) with those fixed numbers.

Take 1,000 students. Each student is one observation, with two features, IQ and CGPA, and a **target** (G-1949; the output we predict): placed (1) or not (0). One student might be IQ 78, CGPA 7.8, placed 1; another IQ 69, CGPA 5.1, placed 0. We want a model that predicts placement for a new student.

Like any ML algorithm, the perceptron works in two stages:

1. **Training:** feed the students one by one (x_1 = IQ, x_2 = CGPA), compare the output with the true label, and adjust w_1, w_2 and b. The goal of training is only these three numbers.
2. **Prediction:** with the learned numbers fixed, compute z for a new student and pass it through the **step function** (G-1889).

Suppose training gave $w_1 = 1$, $w_2 = 2$ and $b = 3$. A new student has IQ 100 and CGPA 5.1. Section 3.2 already computed $z = 113.2$. Since $z \geq 0$, the step function outputs 1: we predict this student will be placed.

Figure 3 runs prediction on real data. This dataset is different from the IQ example above: it is the one used in section 8, with 100 students and two features, CGPA and a resume score. The perceptron was trained there, with both inputs standardized (**standardization**, G-1874: rescaled to mean 0 and standard deviation 1). Its learned numbers are $w_1 = 5.82$, $w_2 = 1.48$ and $b = 1$.

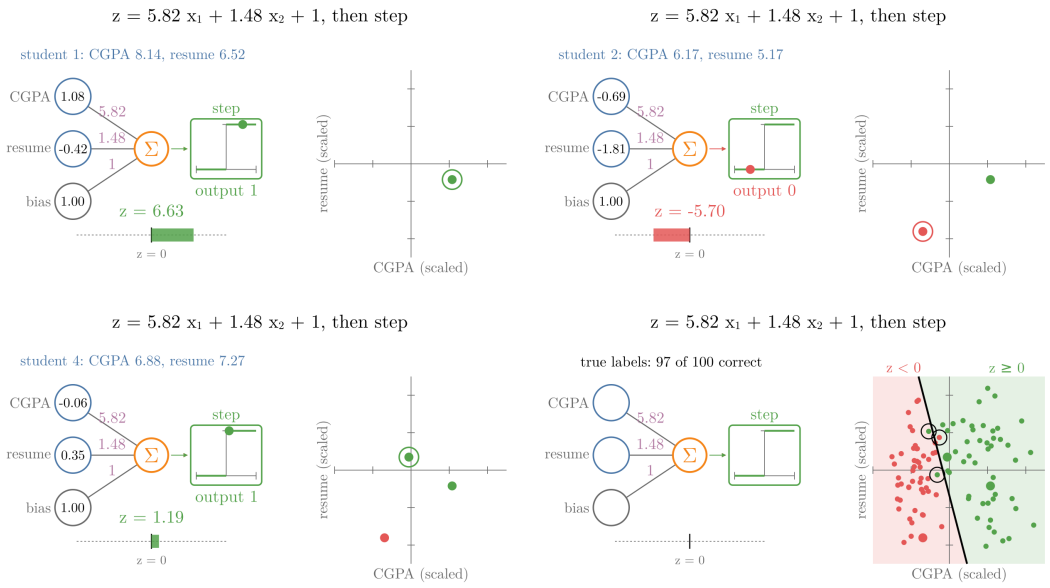


Figure 3: Prediction with a trained perceptron, $z = 5.82x_1 + 1.48x_2 + 1$ on standardized CGPA and resume score. For each student the inputs flow in, z fills a bar (green if positive, red if negative), the step function fires 1 or 0, and the student lands on the plane in that colour. At the end all 100 students appear with the line z = 0 between the two regions

Watch the bar: its sign alone decides the output. Student 4 has $z = 1.19$, just above 0, and lands close to the line. The last frame is the straight **decision boundary** (G-555) of section 7, with 97 of the 100 students on their correct side. The three circled students are the ones on the wrong side.

4.1 More inputs

> **Key point:** One more feature means one more input node and one more weight; nothing else changes.

If we also know each student's state (encoded as a number, see the **one-hot encoding** (G-1379)), we add an input x_3 with weight w_3:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

With n inputs, z is the sum of n products plus the bias. The **summation** (G-1915) and the activation stay as they are.

5. Perceptron and biological neuron

> **Key point:** The **perceptron** (G-1486) is loosely inspired by a brain cell, but it is far simpler and is not a copy of it.

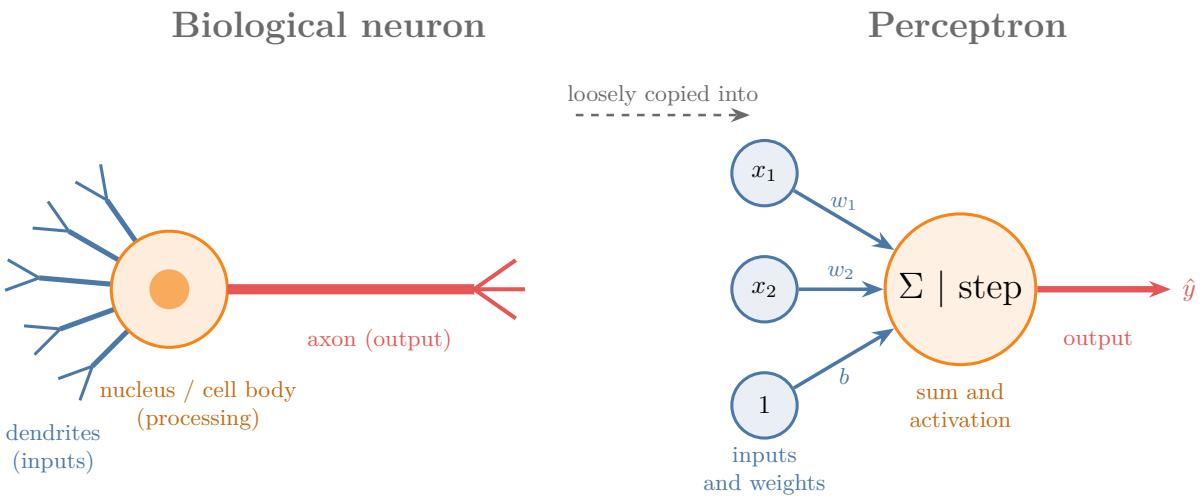


Figure 4: A biological neuron and a perceptron, part by part

The nervous system is a network of billions of **neurons**, the brain cells (**neuron**, G-1318). Figure 4 sets one beside a perceptron:

Neuron	Role	Perceptron
Dendrites	receive signals	inputs and weights
Nucleus (cell body)	processes them	summation and activation
Axon	sends the result on	output

Neurons connect to each other to form the nervous system; perceptrons connect to each other to form a neural network. The similarity ends there.

5.1 Three differences

> **Key point:** A neuron is complex, its processing is not understood, and its connections change over time. A **perceptron** (G-1486) is simple, its processing is two known operations, and its connections are fixed once trained.

1. **Complexity:** a neuron takes a very large number of inputs and is wired to many other neurons. A perceptron has a handful of inputs and one output.
2. **Processing:** inside a neuron, electrochemical reactions decide the output, and scientists still do not fully understand them. Inside a perceptron there are exactly two operations: a **weighted sum** (G-2119) and a step.
3. **Neuroplasticity** (G-1319): the connections between neurons get stronger or weaker over time, disappear, or form anew; this is how we learn new skills. A trained perceptron's connections (its weights) do not change.

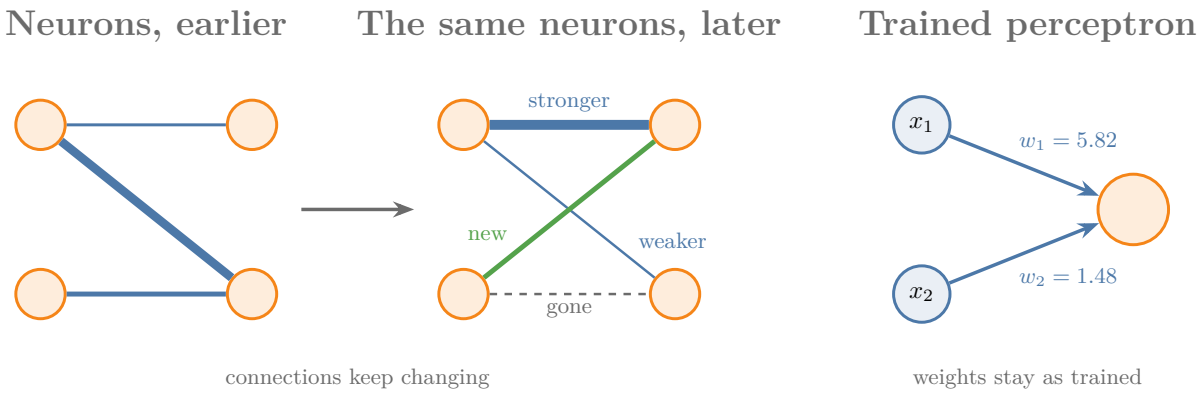


Figure 5: Neuroplasticity: connections between neurons get stronger, get weaker, vanish and appear over time; the weights of a trained perceptron stay fixed

Figure 5 draws the third difference: the arrows between neurons thicken, thin, vanish or appear as time passes, while the perceptron's arrows stay the same.

So the perceptron is weakly inspired by the neuron. Calling it a model of the brain would be wrong.

6. Weights as feature importance

> **Key point:** A bigger weight means a stronger connection: that input has more say in the decision.

Suppose training on the placement data gave $w_1 = 2$ for IQ, $w_2 = 4$ for CGPA and $b = 1$. The weight of CGPA is twice the weight of IQ. So, for this model, CGPA matters about twice as much as IQ in deciding placement. The box below gives another way to see it.

In this way the weights tell us the **feature importance** (G-764) of each input. The **bias** (G-284) does not belong to any input; its role is to shift the line (Section 7).

Figure 6 shows what a weight does to the decision. It takes the trained **perceptron** (G-1486) of section 8, on standardized CGPA (x_1) and resume score (x_2):

$$z = 5.82\,x_1 + 1.48\,x_2 + 1$$

The figure keeps w_1 and b and changes only w_2.

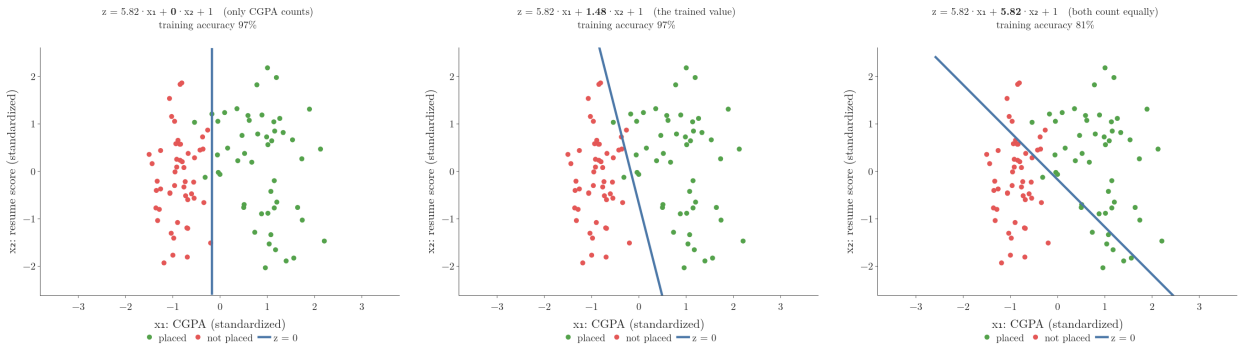


Figure 6: The line z = 0 as the resume-score weight w2 grows from 0 to 8, with w1 = 5.82 and b = 1 fixed, over the 100 students of section 8.

1. $w_2 = 0$**.** The resume score has no say: only CGPA decides, and the line is vertical. Training accuracy is already 97 percent.
2. $w_2 = 1.48$**, the trained value.** The resume score counts a little; the line leans slightly. Accuracy is 97 percent.
3. $w_2 = 5.82$**, equal to** w_1**.** Both inputs count equally, and the line runs diagonally. Accuracy falls to 81 percent, because in this data CGPA matters far more than the resume score.

The bigger a weight compared with the others, the more the line turns to follow that input.

> **Another way to see it:** Reading weights as importance is only fair when the inputs are on the same scale. IQ runs to about 150 while CGPA stops at 10, so a weight on IQ is multiplied by much bigger numbers. Standardize the inputs first (see the **standardization** (G-1874)) and then compare the size of the weights. Section 8 shows a case where the raw weights even get the sign wrong.

7. What a perceptron does geometrically

> **Key point:** z = 0 is a line (a plane in 3D, a hyperplane beyond). The **perceptron** (G-1486) predicts 1 on one side and 0 on the other: it is a binary classifier with one straight decision boundary.

Rename the weights, the **bias** (G-284) and the inputs, one at a time:

$$w_1 \to A, \qquad w_2 \to B, \qquad b \to C$$
$$x_1 \to x, \qquad x_2 \to y$$

Then $z = w_1x_1 + w_2x_2 + b = 0$ becomes

$$Ax + By + C = 0$$

This is the general form of a line (see the **perceptron trick** (G-1485), section 3).

- $z \geq 0$ is the region on the positive side of the line: predict 1, placed.
- $z < 0$ is the region on the negative side: predict 0, not placed.

Figure 7 shows where this line comes from, for the same trained perceptron. It is a 3D picture: the two horizontal axes are the two inputs, and the height is z. Watch how the flat sheet of heights is cut at height 0.

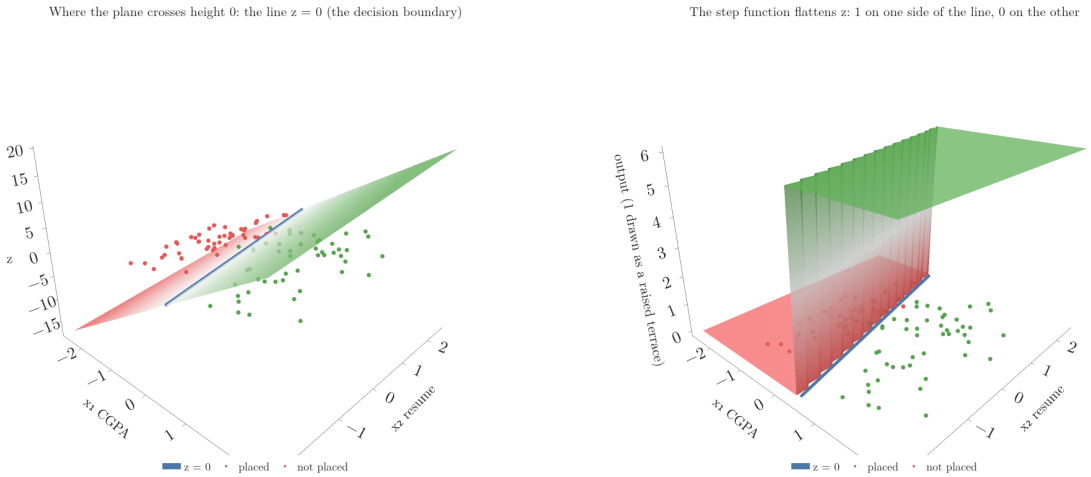


Figure 7: The perceptron of section 8 in 3D. First z is a tilted plane above the students; it crosses height 0 along the line z = 0; then the step function turns z into two flat terraces, 1 and 0.

1. **z is a plane.** For every point of the input plane, the score z gives a height. Together the heights form a tilted plane: high above the placed students, below zero under the others.

2. **The plane crosses zero along a line.** Where the height is exactly 0, we get the line $z = 0$, the blue line.
3. **The step flattens it.** The **step function** (G-1889) replaces every positive height by 1 and every negative height by 0, so the tilted plane becomes two flat terraces that meet at the blue line.

So a trained perceptron is nothing more than a line that splits the plane into two **decision regions** (G-557), one per class. This line is the perceptron's **decision boundary** (G-555). One line gives exactly two regions, so the perceptron is a **binary classifier** (G-302): it separates exactly two classes.

With three inputs the points live in 3D and $z = 0$ is a **plane** (G-1502), like a sheet held across a room. With four or more inputs it is a **hyperplane** (G-911; see the **equation of a hyperplane** (G-703)). In every case there are two regions.

7.1 The big limitation

> **Key point:** A **perceptron** (G-1486) only works on data that a straight decision boundary can (roughly) separate.

A perceptron can only classify linearly separable (G-1103) data, or data that is almost so: a few points on the wrong side only lower the accuracy. If one class surrounds the other, no line can split them and the perceptron fails, whatever we do. The problem with the perceptron shows this in code.

8. A perceptron in scikit-learn

> **Key point:** scikit-learn's **Perceptron** (G-1486) learns w_1, w_2 and b with fit; coef_ holds the weights and intercept_ the bias.

The data has 100 students with two inputs, CGPA and a resume score (both from 0 to 10), and the output `placed`. Fifty were placed and fifty not.

Python: Training a perceptron.

```
import pandas as pd
from sklearn.linear_model import Perceptron

df = pd.read_csv("data/placement.csv")
X = df[["cgpa", "resume_score"]]
y = df["placed"]

p = Perceptron(random_state=0)
p.fit(X, y)
p.coef_        # [[40.26, -36.0]]: w1 (CGPA), w2 (resume score)
p.intercept_   # [-25.0]: the bias b
```

`Perceptron` lives in `sklearn.linear_model`, next to `LogisticRegression`.

> `random_state` fixes the order in which it visits the observations.

So the learned line is the set of points where

$$40.26\,x_1 - 36\,x_2 - 25 = 0$$

Here x_1 is CGPA and x_2 is resume score.

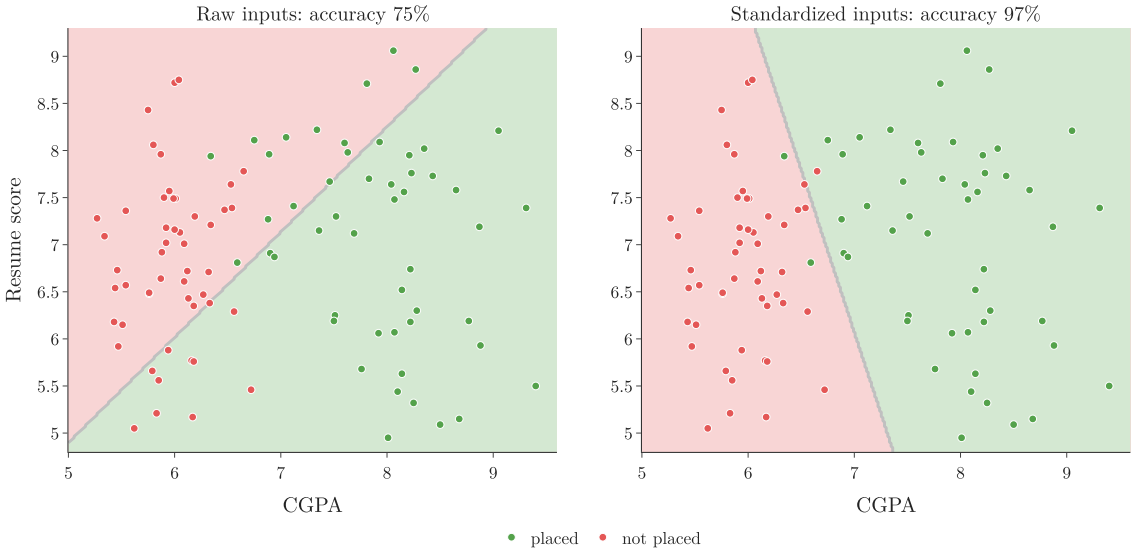


Figure 8: The perceptron's line and its two regions, before and after standardizing the inputs

Figure 8 (left) colours each region by the class the perceptron predicts there: green where it predicts placed, red where it predicts not placed. The grey line is the **decision boundary** (G-555), where the prediction switches from one class to the other. These regions are not a loss surface and need no contour reading. The line divides the data, but badly: the training accuracy is only 75%. The weight on resume score is even negative, which would mean a better resume lowers the chance of placement.

The fix is to standardize the inputs first (Figure 8, right). Training accuracy rises to 97%, and the weights become 5.82 for CGPA and 1.48 for resume score. On scaled inputs the weights also make sense as **feature importance** (G-764): CGPA counts about four times as much as the resume score.

Why raw inputs hurt here: each training step moves the weights by about 7 (the size of a CGPA or a resume score) but the **bias** (G-284) by only 1, because the bias's input is the constant 1. This data needs a large bias, so the bias is like a walker taking baby steps beside two runners: training ends before the bias gets where it needs to be.

Extra: The evidence, from the Notebook, changing one thing at a time.

- Both features already share a scale (each runs from about 5 to 9.5), so the size of the numbers is not the problem; their distance from the origin is. The observations sit around $(6.9, 6.9)$.
- Subtracting the mean alone gives 96%. Dividing by the standard deviation alone gives 50%.
- On raw inputs, scikit-learn's early stop (**early stopping**, G-656) ends training after 7 **epochs** (G-696; full passes over the training data): it stops once the loss has not improved for `n_iter_no_change=5` epochs in a row (scikit-learn docs, Perceptron).
- Turning the early stop off and training for 1,000 epochs on raw inputs reaches 92%, with a bias of -673. The bias does reach a large value, but only after many more epochs.

The data here is small and nothing was tuned, so the raw result is not the best a perceptron can do. The aim is to see the three learned numbers and the line they describe.

Python: Drawing the regions. The figure colours a fine grid of points by `p.predict`, as in the toy project (Figure 7). The Notebook has the code; it uses Plotly instead of the `mlxtend` library's `plot_decision_regions`.

9. Summary

Part	What it does	In the placement example	Why
Inputs x_1, x_2	the observation's feature values	CGPA, resume score	one input node per feature
Weights w_1, w_2	how much each input counts	learned on raw inputs: 40.26, -36; on standardized inputs: 5.82, 1.48	they set the direction of the line; a bigger weight gives its input more say
Bias b	shifts the line	learned on raw inputs: -25; on standardized inputs: 1	without it the line could only pass through the origin
Summation	$z = w_1x_1 + w_2x_2 + b$	one number per student	the sign of this one number decides the output
Activation (step)	1 if $z \geq 0$, else 0	placed or not	it turns a score of any size into a class

- A **perceptron** (G-1486) is a weighted sum plus a bias, followed by an activation function, so a trained perceptron is fully described by its weights and bias.
- The bias is a threshold: the output is 1 when the **weighted sum** (G-2119) is at least $-b$. Changing the **bias** (G-284) moves the line without turning it.
- Training finds the weights and bias; prediction applies them to a new observation, so after training these few numbers are all the perceptron needs.
- It is loosely inspired by a neuron (dendrites, nucleus, axon) but far simpler, and its weights are fixed once trained, so calling it a model of the brain would be wrong.
- On standardized inputs, larger weights mean more important inputs, because on raw inputs a weight on a large-scale feature (IQ up to about 150) multiplies much bigger numbers than one on CGPA.
- Geometrically it is a line, plane or hyperplane: a binary classifier for **linearly separable** (G-1103) data only, because one straight boundary gives exactly two regions, and no line can split a class that surrounds another.
- So a perceptron weighs the inputs, adds a bias and steps the result, which draws one straight boundary with each class on its own side.

10. Sources

Built from

- CampusX, "What is a Perceptron? Perceptron Vs Neuron | Perceptron Geometric Intuition", YouTube, https://www.youtube.com/watch?v=X7iIKPoZ0Sw
- Sanderson, G. (3Blue1Brown), "But what is a neural network? | Deep learning chapter 1", 2017, https://www.youtube.com/watch?v=aircAruvnKk (the bias as the level the

weighted sum must pass)

Other references

- scikit-learn developers, *sklearn.linear_model.Perceptron*, API reference, scikit-learn.org (parameters `tol`, `n_iter_no_change`, `max_iter`).

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Activation function (G-165)	The function that turns a node's weighted sum z into its output, bringing it into a fixed range.
Perceptron (G-1486)	The smallest building block of a neural network, one artificial neuron: it multiplies each input by a weight, adds the results and turns the sum into an output.
Weight (in a network) (G-2106)	A number the network learns on the connection between two neurons; it sets how strongly one neuron's output counts in the next neuron's sum.
Bias (of a perceptron) (G-284)	The weight on a constant input of 1; it shifts the boundary away from the origin.
Summation (z) (G-1915)	The weighted sum $w_1x_1 + w_2x_2 + \cdots + b$ inside a perceptron.
Threshold (of a perceptron) (G-2261)	The value the weighted sum must reach for the output to be 1; it equals $-b$.
Neuron (biological) (G-1318)	A brain cell: dendrites take signals in, the nucleus processes them, the axon sends the result on.
Neuroplasticity (G-1319)	The brain's connections strengthening, weakening, vanishing or forming over time.
Binary classifier (G-302)	A model that separates exactly two classes.
Dendrites, nucleus, axon (G-581)	The parts of a brain neuron that receive signals (dendrites), process them (nucleus) and send the result on (axon); in a perceptron they match the inputs with weights, the summation and activation, and the output.
Feature importance (weights) (G-763)	Reading the size of a weight as how much its input matters, fair only on scaled inputs.

Term	Meaning
Neural network (G-1316)	A model made of many simple connected units (neurons) in layers, loosely inspired by the brain; it learns the weights of its connections from data and is the model deep learning uses.
Supervised learning (G-1919)	Learning from data with inputs and outputs, to predict outputs.
Step function (G-1889)	A function that outputs 1 for positive inputs and 0 otherwise; the classic perceptron uses it to turn its score into a class.
Perceptron trick (G-1485)	Moving a line towards each misclassified point until the classes are separated.
Plane (G-1502)	A flat surface in 3D; the model for two input columns.
Equation of a hyperplane (G-703)	One equation for a flat boundary in any number of dimensions, a line in 2D, a plane in 3D, a hyperplane beyond: $w^\mathsf{T}x + w_0 = 0$.
Dot product (G-634)	Multiplying two vectors entry by entry and adding the products, $u^\mathsf{T}x$, which gives one number. It is large when the vectors point the same way, so it measures similarity and gives projections.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Parameter (of a function) (G-1448)	A named setting passed to a function, like `sep=";"`.
Weighted sum (G-2119)	In self-attention, the value vectors multiplied by their weights and added together; the result is the word's new, context-aware vector.
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Standardization (G-1874)	Scaling a column to mean 0 and standard deviation 1 (subtract the mean, divide by the standard deviation), so features measured on different scales become comparable.
Decision boundary (G-555)	A line or curve that separates the classes in classification.
One-hot encoding (G-1379)	Replacing a nominal column by one 0/1 column per category, with a single 1 in each row; also used to represent words.

Term	Meaning
Feature importance (G-764)	How much a column helped a tree: its share of all the impurity reduction (how much purer the nodes became) achieved by the tree's splits; the shares add up to 1. It shows which columns the tree relied on and helps with feature selection.
Decision region (G-557)	The part of the input space in which a model predicts a given class.
Hyperplane (G-911)	A flat surface in more than three dimensions; the model for three or more input columns.
Linearly separable (G-1103)	Data whose classes a straight line, plane or hyperplane can split.
Early stopping (G-656)	Stopping training when the score on held-out data is best, before full convergence; this keeps coefficients small, so it also acts as regularisation.
Epoch (G-696)	One full update of the parameters using the whole training set.